

PostgreSQL Data Persistence Using Docker Volumes

1. Introduction

This project demonstrates how Docker **named volumes** can be used to persist PostgreSQL database data even after stopping and removing containers. The implementation is performed on an **Ubuntu system**, and screenshots can be captured at each step for documentation and evaluation purposes.

2. Objective

- To run a PostgreSQL database inside a Docker container
- To store database data persistently using Docker volumes
- To verify data persistence after container removal
- To understand volume lifecycle management in Docker

3. Prerequisites

- Ubuntu OS
- Docker installed and running
- Basic knowledge of Docker and PostgreSQL

4. Project Architecture

- **Docker Image:** postgres (official)
- **Container Names:** db, new-db
- **Database User:** postgres
- **Volume Name:** postgres_data
- **Database Tool:** psql CLI

5. Step-by-Step Implementation

Step 1: Run PostgreSQL Container with Volume Mapping

```
docker run --name=db \  
-e POSTGRES_PASSWORD=secret \  
-d \  
-v postgres_data:/var/lib/postgresql/data \  
postgres
```

Explanation:

- Starts PostgreSQL in detached mode
- Sets database password
- Attaches a named volume for persistent storage

```
admin@RitejMachine:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
6bf91e913008	postgres:18	"docker-entrypoint.s..."	4 minutes ago	Up 4 minutes
de47c776bed4	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	postgres	14 minutes ago	Exited (1) 14 minutes ago
0059c43c9fb6	b5caf683a8bb	"docker-entrypoint.s..."	27 minutes ago	Exited (1) 15 minutes ago
75033139141f	postgres	"docker-entrypoint.s..."	5 days ago	Exited (255) 4 days ago
859ad8f76785	getting-started	"docker-entrypoint.s..."	5 days ago	Exited (255) 5 days ago
	127.0.0.1:3000->3000/tcp	recursing_kare		
	flask	"python app.py"		
	0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp	ritesh		

Step 2: Connect to PostgreSQL Database

```
docker exec -it db psql -U postgres
```

Explanation:

- Opens PostgreSQL interactive shell inside the container

```
admin@RitejMachine:~$ docker exec -ti 6bf91e913008 psql -U postgres
psql (18.1 (Debian 18.1-1.pgdg13+2))
Type "help" for help.
```

Step 3: Create Table and Insert Records

```
CREATE TABLE tasks (
  id SERIAL PRIMARY KEY,
  description VARCHAR(100)
);
```

```
INSERT INTO tasks (description)
VALUES ('Finish work'), ('Have fun');
```

Explanation:

- Creates a table named tasks
- Inserts two sample records

```
admin@RitejMachine:~$ docker exec -ti 6bf91e913008 psql -U postgres
psql (18.1 (Debian 18.1-1.pgdg13+2))
Type "help" for help.

postgres=# CREATE TABLE tasks (
id SERIAL PRIMARY KEY,
description VARCHAR(100)
);
CREATE TABLE
postgres=# SELECT * FROM tasks;
 id | description
----+-----
(0 rows)
```

Step 4: Verify Inserted Data

```
SELECT * FROM tasks;
```

Expected Output:

```
id | description
---+-----
1  | Finish work
2  | Have fun
```

```
postgres=# INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');
INSERT 0 2
postgres=# SELECT * FROM tasks;
 id | description
----+-----
 1  | Finish work
 2  | Have fun
 3  | Finish work
 4  | Have fun
(4 rows)
```

Step 5: Exit PostgreSQL Shell

```
\q
postgres=# \q
admin@RitejMachine:~$
```

Step 6: Stop and Remove Database Container

```
docker stop db
docker rm db
```

Explanation:

- Stops and removes the PostgreSQL container
- Volume remains intact

```
admin@RitejMachine:~$ docker stop postgres
postgres
admin@RitejMachine:~$ docker rm postgres
postgres
admin@RitejMachine:~$
```

Step 7: Start New Container Using Same Volume

```
docker run --name=new-db \
-d \
-v postgres_data:/var/lib/postgresql/data \
postgres
```

Explanation:

- Creates a new container
- Reuses existing database volume
- No password variable needed (already initialized)

```
admin@RitejMachine:~$ docker run --name new_db -e POSTGRES_PASSWORD=mysecretpassword -p
5432:5432 -v pgdata:/var/lib/postgresql -d postgres:18
e3a0f3154c633c4cb42f4e7de1be3ae994e0ae7a901197592f2a03878753f150
```

Step 8: Verify Persisted Data

```
docker exec -it new-db psql -U postgres -c "SELECT * FROM tasks"
```

Expected Output:

```
id | description
---+-----
1 | Finish work
2 | Have fun
admin@RitejMachine:~$ docker exec -it e3a0f3154c63 psql -U postgres -c "SELECT * FROM ta
sks"
 id | description
---+-----
  1 | Finish work
  2 | Have fun
```

6. Volume Cleanup

Remove Container Using the Volume

```
docker rm -f new-db
```

Remove Specific Volume

```
docker volume rm postgres_data
```

Remove All Unused Volumes

```
docker volume prune
admin@RitejMachine:~$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
21eadc863bc029ddb3df85d44eafe79708ba83f83776d28b4827d0e81ee222b1
c94cdc865b2a6b9ea8874c58cb81d209aa7482b9d3401435be1e1e53a8c1ed16

Total reclaimed space: 0B
admin@RitejMachine:~$ |
```

7. Result

The PostgreSQL database data was successfully persisted using a Docker named volume. Even after removing the original container, the data remained intact and accessible when attached to a new container.

8. Conclusion

This project proves that Docker volumes are an effective solution for database persistence. Using named volumes ensures data safety, portability, and container independence, which is essential for production-grade containerized applications.

9. Key Learnings

- Docker volumes enable persistent storage
- Environment variables are required only during initial database setup
- Containers are ephemeral, volumes are persistent
- PostgreSQL works seamlessly with Docker volumes